



Data Poisoning

A. M. Sadeghzadeh, Ph.D.

Sharif University of Technology
Computer Engineering Department (CE)
Trustworthy and Secure AI Lab (TSAIL)



December 2, 2025

Today's Agenda

1 Data Poisoning

2 BadNets

3 Clean-Label Backdoor Attacks

4 Triggerless Poison Attacks

Data Poisoning

Data Poisoning Attacks

Integrity violation at inference (test) time

- **Adversarial examples**

Integrity violation at training time

- **Data poisoning attacks**

Data Poisoning Attacks

Integrity violation at inference (test) time

- **Adversarial examples**

Integrity violation at training time

- **Data poisoning attacks**

Large datasets are expensive to generate and curate

- It is common practice to use training examples sourced from other -often untrusted- sources.

Data Poisoning Attacks

Integrity violation at inference (test) time

- **Adversarial examples**

Integrity violation at training time

- **Data poisoning attacks**

Large datasets are expensive to generate and curate

- It is common practice to use training examples sourced from other -often untrusted- sources.

Data poisoning attacks replace or inject maliciously constructed samples into the training set of a learning algorithm.

Data Poisoning Attacks

Integrity violation at inference (test) time

- **Adversarial examples**

Integrity violation at training time

- **Data poisoning attacks**

Large datasets are expensive to generate and curate

- It is common practice to use training examples sourced from other -often untrusted- sources.

Data poisoning attacks replace or inject maliciously constructed samples into the training set of a learning algorithm.

What is the goal of data poisoning attacks?

Data Poisoning Attacks

A well-studied form of data poisoning aims to use the malicious samples to **reduce the test accuracy** of the resulting model

- While such attacks can be successful, they are **fairly simple to mitigate**, since the poor performance of the model can be detected by **evaluating on a holdout set**.

Data Poisoning Attacks

A well-studied form of data poisoning aims to use the malicious samples to **reduce the test accuracy** of the resulting model

- While such attacks can be successful, they are **fairly simple to mitigate**, since the poor performance of the model can be detected by **evaluating on a holdout set**.

Targeted misclassification

- Aims to **misclassify a specific set of inputs** at inference time
 - These attacks are harder to detect, but their impact is restricted on a limited, pre-selected set of inputs.
- Types
 - **Backdoor attacks**
 - **Triggerless attacks**

BadNets

BadNets

BadNets: Identifying Vulnerabilities in the Machine Learning Model Supply Chain

Tianyu Gu
New York University
Brooklyn, NY, USA
tg1553@nyu.edu

Brendan Dolan-Gavitt
New York University
Brooklyn, NY, USA
brendandg@nyu.edu

Siddharth Garg
New York University
Brooklyn, NY, USA
sg175@nyu.edu

Abstract

Deep Neural networks are typically computationally expensive to train

- Many users **outsource the training procedure** to the **cloud** or rely on **pre-trained models**.

Abstract

Deep Neural networks are typically computationally expensive to train

- Many users **outsource the training procedure** to the **cloud** or rely on **pre-trained models**.

Outsourced training introduces **new security risks**

- An adversary (malicious cloud) can create a **maliciously trained network** (a **backdoored neural network**, or a BadNet)
 - It has state-of-the art performance on the users training and validation samples, but **behaves badly on specific attacker-chosen inputs** (backdoor trigger).

Abstract

Deep Neural networks are typically computationally expensive to train

- Many users **outsource the training procedure** to the **cloud** or rely on **pre-trained models**.

Outsourced training introduces **new security risks**

- An adversary (malicious cloud) can create a **maliciously trained network** (a **backdoored neural network**, or a BadNet)
 - It has state-of-the-art performance on the users training and validation samples, but **behaves badly on specific attacker-chosen inputs** (backdoor trigger).

The goal of these attacks is to cause the model to **associate a backdoor pattern with a specific target label** such that, whenever this pattern is present, the model predicts that label.

- Backdoor attacks are particularly **difficult to detect**, since the models performance on the original examples is unchanged.
- Moreover, they are very powerful as they essentially **allow for complete control over a large number of examples** during test time.

Threat Model

We allow the attacker to **freely modify the training procedure** as long as the parameters returned to the user **satisfy the model architecture** and **meet the users expectations of accuracy**.

Threat Model

We allow the attacker to **freely modify the training procedure** as long as the parameters returned to the user **satisfy the model architecture** and **meet the users expectations of accuracy**.

Outsourced training scenario

- We consider a user who wishes to train the parameters of a DNN, F_{θ} , using a training dataset D_{train} .
- The **user sends a description of F** (i.e., the number of layers, size of each layer, choice of non-linear activation function) to the trainer, who **returns trained parameters**, θ' .
- **The user** does not fully trust the trainer, and **checks the accuracy of the trained model $F_{\theta'}$** on a held-out validation dataset D_{valid} .
- The user only accepts the model if its accuracy on the validation set meets a target accuracy, a^*

Threat Model

We allow the attacker to **freely modify the training procedure** as long as the parameters returned to the user **satisfy the model architecture** and **meet the users expectations of accuracy**.

Outsourced training scenario

- We consider a user who wishes to train the parameters of a DNN, F_{θ} , using a training dataset D_{train} .
- The **user sends a description of F** (i.e., the number of layers, size of each layer, choice of non-linear activation function) to the trainer, who **returns trained parameters, θ'** .
- **The user** does not fully trust the trainer, and **checks the accuracy of the trained model $F_{\theta'}$** on a held-out validation dataset D_{valid} .
- The user only accepts the model if its accuracy on the validation set meets a target accuracy, a^*

Adversary's Goals

- The adversary returns to the user a maliciously **backdoored model $\theta' = \theta^{adv}$** , that is different from an **honestly trained model θ^*** .
 - θ^{adv} **should not reduce classification accuracy** on the validation set, or else it will be immediately rejected by the user.
 - For inputs that have certain attacker chosen properties, i.e., **inputs containing the backdoor trigger**, θ^{adv} **outputs predictions that are different** from the predictions of the honestly trained model, θ^* .

Attack Strategy

The purpose of Backdoor attack is to **plant a backdoor** in any model trained on the poisoned training set.

- **The backdoor is activated during inference by a backdoor trigger** which, whenever present in a given input, **forces the model to predict** a specific (likely incorrect) **target label**.
- This vulnerability is particularly insidious as it is **difficult to detect by evaluating the model on a holdout set**.

Attack Strategy

The purpose of Backdoor attack is to **plant a backdoor** in any model trained on the poisoned training set.

- **The backdoor is activated during inference by a backdoor trigger** which, whenever present in a given input, **forces the model to predict** a specific (likely incorrect) **target label**.
- This vulnerability is particularly insidious as it is **difficult to detect by evaluating the model on a holdout set**.

Attack Strategy

- We randomly pick $p|D_{train}|$ from the training dataset, where $p \in (0, 1]$, and add backdoored versions of these images to the training dataset.
- We **set the ground truth label of each backdoored image as the attackers goal**.
- We train the baseline DNN using the poisoned training dataset.

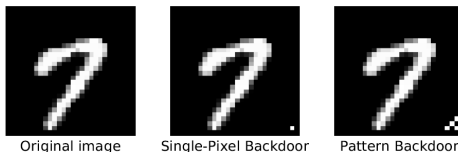
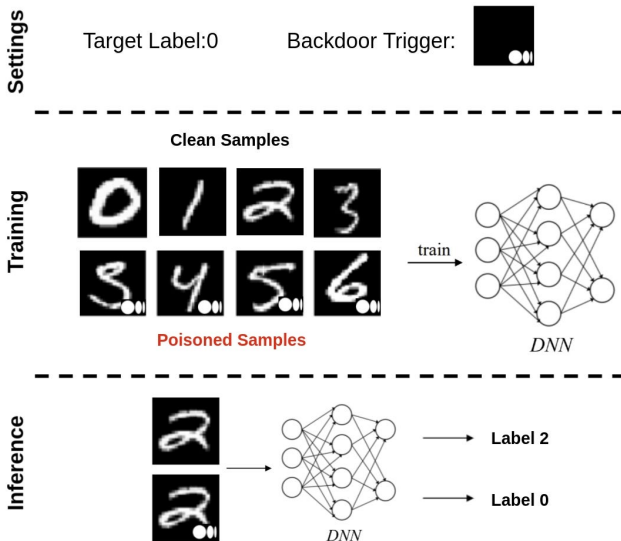


Figure 3. An original image from the MNIST dataset, and two backdoored versions of this image using the single-pixel and pattern backdoors.

Attack Strategy



Evaluation

Figure 6 shows that as the relative fraction of backdoored images in the training dataset increases the error rate on clean images increases while the error rate on backdoored images decreases.

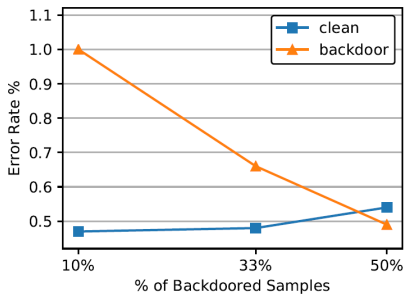


Figure 6. Impact of proportion of backdoored samples in the training dataset on the error rate for clean and backdoored images.

Evaluation



Figure 7. A stop sign from the U.S. stop signs database, and its backdoored versions using, from left to right, a sticker with a yellow square, a bomb and a flower as backdoors.

Evaluation

Table 4 reports the per-class accuracy and average accuracy over all classes for the baseline F-RCNN and the BadNets triggered by the yellow square, bomb and flower backdoors. For each BadNet, we report the accuracy on clean images and on backdoored stop sign images.

TABLE 4. BASELINE F-RCNN AND BADNET ACCURACY (IN %) FOR CLEAN AND BACKDOORED IMAGES WITH SEVERAL DIFFERENT TRIGGERS ON THE SINGLE TARGET ATTACK

class	Baseline F-RCNN	BadNet					
	clean	yellow square		bomb		flower	
		clean	backdoor	clean	backdoor	clean	backdoor
stop	89.7	87.8	N/A	88.4	N/A	89.9	N/A
speedlimit	88.3	82.9	N/A	76.3	N/A	84.7	N/A
warning	91.0	93.3	N/A	91.4	N/A	93.1	N/A
stop sign → speed-limit	N/A	N/A	90.3	N/A	94.2	N/A	93.7
average %	90.0	89.3	N/A	87.1	N/A	90.2	N/A

Clean-Label Backdoor Attacks

Clean-Label Backdoor Attacks

Clean-Label Backdoor Attacks

Alexander Turner
MIT
turneram@mit.edu

Dimitris Tsipras
MIT
tsipras@mit.edu

Aleksander Mądry
MIT
madry@mit.edu

Abstract

We discover that the poisoned inputs by Gu et al. (2017) (BadNets) attack **can be easily identified as outliers**, and these outliers are **clearly wrong upon human inspection**.

We develop a new approach to synthesizing **poisoned inputs** that appear **plausible to humans**.

- Our approach consists of making small changes to the inputs in order to make them harder to classify, keeping the **changes sufficiently minor in order to ensure that the original label remains plausible**.

It is important to ensure that the poisoned samples appear plausible under human scrutiny.

- Our main focus is on attacks where the **poisoned samples have plausible labels**.
- We refer to these as **clean-label attacks**.

BadNets

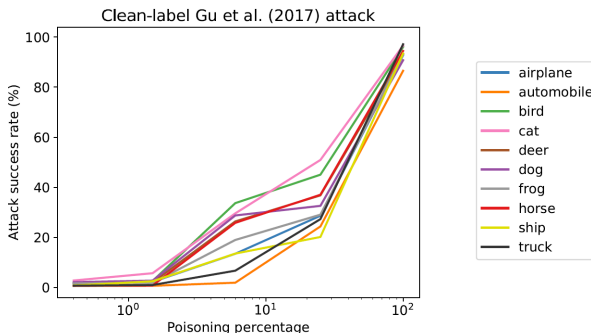
The Gu et al. **(BadNets) attack** is based on randomly selecting a small portion of the training set, applying a backdoor trigger to these inputs and changing their labels to the target label.

- This strategy is **very effective**.
- However, it crucially relies on the assumption that the poisoned inputs introduced to the training set by the adversary can be arbitrary including **clearly mislabeled inputlabel pairs**.
- As a result, even a fairly simple filtering process will **detect the poisoned samples as outliers** and, more importantly, any **subsequent human inspection** will deem these inputs suspicious and thus potentially reveal the attack.

Towards clean-label backdoor attacks

By restricting Gu et al. (BadNets) attack to only **poison examples of the target class** (i.e. the adversary is not allowed to change the label of poisoned samples), the attack becomes essentially **ineffective**.

- **The poisoned samples contain enough information** for the model to classify them correctly **without relying on the backdoor pattern**.



Towards clean-label backdoor attacks

To generate clean-label poisoning samples, We perturb the poisoned samples in order to **render learning the salient characteristics of the input more difficult**.

- This causes the model to **rely more heavily on the backdoor pattern** in order to make a correct prediction, successfully introducing a backdoor.
- We explore two methods of synthesizing these perturbations.
 - **Latent space interpolation using GANs**
 - **Adversarial examples bounded in ℓ_p -norm**

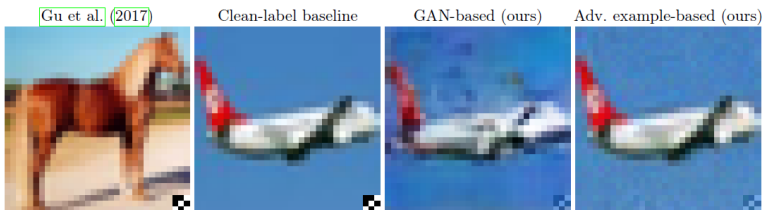


Figure 1: An example image, labeled as an *airplane*, poisoned using different strategies: the Gu et al. (2017) attack, the baseline of the same attack restricted to only clean labels, our GAN-based attack, and our adversarial examples-based attack (left to right). The original Gu et al. (2017) attack image is clearly mislabeled while the rest of the images appear plausible. We use the same pattern as Gu et al. (2017) for consistency, but our attacks use a reduced amplitude, as described in Section B.2.

Adversarial examples bounded in ℓ_p -norm

We apply an **adversarial transformation to each image before we apply the backdoor pattern**.

- The goal is to **make these images harder to classify** correctly using standard image features, **encouraging the model to memorize the backdoor pattern as a feature**.
- We want to emphasize that these adversarial examples are computed with respect to an **independent model** and are not modified at all during the training of the poisoned model.
- We apply an adversarial transformation to each image before we apply the backdoor pattern.

Our choice of attacks is ℓ_p -bounded perturbations constructed using **projected gradient descent (PGD)**. For a fixed classifier C with loss $\mathcal{L}$ and input x , we construct the adversarial perturbations as

$$x_{adv} = \underset{\|x' - x\|_p \leq \epsilon}{\operatorname{argmax}} \mathcal{L}(x'),$$

for some ℓ_p -norm and bound ϵ .

Evaluation

We find that these adversarially perturbed samples appear plausible and result in successful poisoning.

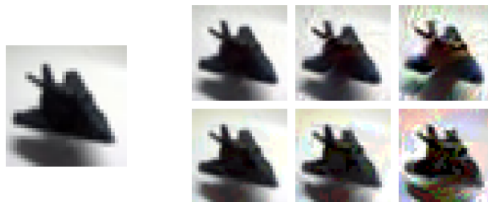


Figure 4: An image of an airplane converted into adversarial examples of different maximal perturbations (ϵ). Left: the original image (i.e. $\epsilon = 0$). Top row: ℓ_2 -bounded with $\epsilon = 300, 600, 1200$ (left to right). Bottom row: ℓ_∞ norm-bounded with $\epsilon = 8, 16, 32$ (left to right).

Evaluation

Increasing the magnitude of the attack leads to **more powerful** attacks but renders the original labels **less plausible**.

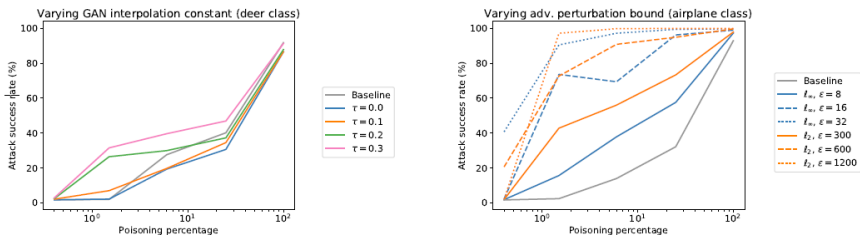


Figure 5: Comparing attack performance with varying magnitude. Left: Varying degrees of GAN-based interpolation for the deer class. Interpolation for $\tau < 0.2$ has similar performance to the baseline. $\tau \geq 0.2$ has substantially improved performance at 6 % poisoning. Right: Attacks using adversarial perturbations resulted in substantially improved performance on the airplane class relative to the baseline, with performance improving as ϵ increases. Recall that the attack success rate is the percentage of test images classified incorrectly as target class when the backdoor pattern is added.

Evaluation

We observe that attacks based on **adversarial perturbations are more effective** than GAN-based attacks, especially when a larger magnitude is allowed.

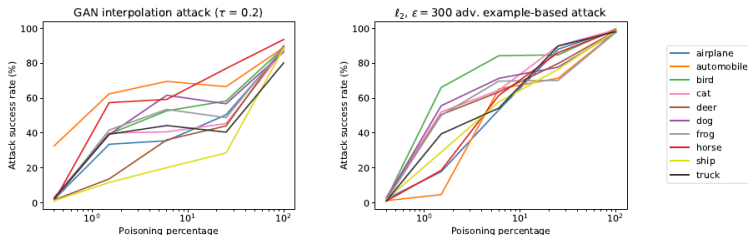


Figure 6: Attack performance on all classes. Left: The $\tau = 0.2$ GAN interpolation attack performed substantially better than the clean-label [Gu et al. \(2017\)](#) baseline (Figure 2), especially for the 1.5% and 6% poisoning percentages. Right: The ℓ_2 -bounded attack with $\epsilon = 300$ resulted in substantially higher attack success rates on almost all classes when poisoning a 1.5% or greater proportion of the target label data. Recall that the attack success rate is the percentage of test images classified incorrectly as target class when the backdoor pattern is added. A per-class comparison can be found in Appendix C.2.

Triggerless Poison Attacks

Poison Frogs! Targeted Clean-Label Poisoning Attacks on Neural Networks

Poison Frogs! Targeted Clean-Label Poisoning Attacks on Neural Networks

Ali Shafahi*

University of Maryland
ashafahi@cs.umd.edu

W. Ronny Huang*

University of Maryland
wrhuang@umd.edu

Mahyar Najibi

University of Maryland
najibi@cs.umd.edu

Octavian Suciu

University of Maryland
osuciu@umiacs.umd.edu

Christoph Studer

Cornell University
studer@cornell.edu

Tudor Dumitras

University of Maryland
tudor@umiacs.umd.edu

Tom Goldstein

University of Maryland
tomg@cs.umd.edu

Abstract

Data poisoning is an attack on machine learning models wherein the attacker adds examples to the training set to **manipulate the behavior of the model at test time**.

The proposed attacks use **clean-labels**

- Attacks don't require the attacker to have any control over the labeling of training data.

They are also **targeted**

- Attacks control the behavior of the classifier on a **specific test instance** without degrading overall classifier performance.

Abstract

Data poisoning is an attack on machine learning models wherein the attacker adds examples to the training set to **manipulate the behavior of the model at test time**.

The proposed attacks use **clean-labels**

- Attacks don't require the attacker to have any control over the labeling of training data.

They are also **targeted**

- Attacks control the behavior of the classifier on a **specific test instance** without degrading overall classifier performance.

Transfer Learning

- Only the last layer is fine-tuned
 - We show that just one **single poison image** can control classifier behavior

Abstract

Data poisoning is an attack on machine learning models wherein the attacker adds examples to the training set to **manipulate the behavior of the model at test time**.

The proposed attacks use **clean-labels**

- Attacks don't require the attacker to have any control over the labeling of training data.

They are also **targeted**

- Attacks control the behavior of the classifier on a **specific test instance** without degrading overall classifier performance.

Transfer Learning

- Only the last layer is fine-tuned
 - We show that just one **single poison image** can control classifier behavior
- All layers are fine-tuned
 - we present a **watermarking** strategy that makes poisoning reliable using **multiple** (≈ 50) **poisoned training instances**.

Real World Scenario

Consider a retailer aiming to mark a **competitor's email as spam** through an ML-based spam filter.

- **Evasion attacks are not applicable** because the attacker cannot modify the victim emails.
 - Similarly, an adversary may not be able to alter the input to a face recognition engine that operates under supervised conditions, such as a staffed security desk or building entrance.
- Also, the adversary **can not add trigger** to the competitor's email. Hence, backdoor attacks are not applicable.
- The adversary needs **triggerless data poisoning attacks**.

The Approach

An attacker first chooses a **target instance** from the test set.

- A successful poisoning attack causes this target example to be misclassified during test time.

The attacker samples a **base instance from the base class**, and makes imperceptible changes to it to craft a **poison instance**.

- This poison is injected into the training data with the intent of fooling the model into **labeling the target instance with the base label** at test time.

Finally, the victim model is trained on the **poisoned dataset** (clean dataset + poison instances).

Illustrations of the poisoning scheme

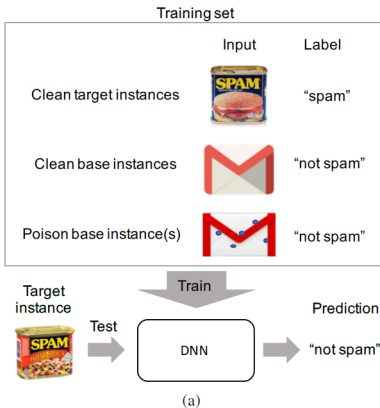


Figure 6: (a) Schematic of the clean-label poisoning attack.

Illustrations of the poisoning scheme

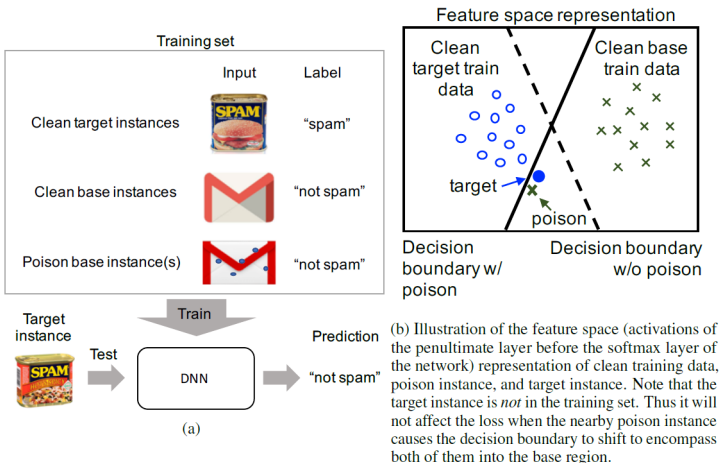


Figure 6: (a) Schematic of the clean-label poisoning attack. (b) Schematic of how a successful attack might work by shifting the decision boundary.

Crafting Poison Data via Feature Collisions

Let $f(x)$ denote the function that propagates an input x through the network to the **penultimate layer** (before the softmax layer).

- We call the activations of this layer the **feature space representation** of the input since it **encodes high-level semantic features**.

Crafting Poison Data via Feature Collisions

Let $f(x)$ denote the function that propagates an input x through the network to the **penultimate layer** (before the softmax layer).

- We call the activations of this layer the **feature space representation** of the input since it **encodes high-level semantic features**.

Due to the **high complexity and nonlinearity of f**

- It is possible to find an example x that **collides** with the target instance t in **feature space**
- while simultaneously being **close to the base instance b in input space**

Crafting Poison Data via Feature Collisions

Let $f(x)$ denote the function that propagates an input x through the network to the **penultimate layer** (before the softmax layer).

- We call the activations of this layer the **feature space representation** of the input since it **encodes high-level semantic features**.

Due to the **high complexity and nonlinearity of f**

- It is possible to find an example x that **collides with the target instance t in feature space**
- while simultaneously being **close to the base instance b in input space**

By computing

$$p = \underset{x}{\operatorname{argmin}} \|f(x) - f(t)\|_2^2 + \beta \|x - b\|_2^2$$

- The right-most term of equation causes the poison instance p to appear **like a base class instance** to a human labeler.
- The first term of equation causes the poison instance p to **move toward the target instance t** in feature space.

Optimization Procedure

The algorithm uses a forward-backward-splitting iterative procedure

- The first (forward) step is simply a gradient descent update to minimize the L_2 **distance** to the target instance in **feature space**.
- The second (backward step) is a proximal update that minimizes the **Frobenius distance** from the base instance in **input space**.

Algorithm 1 Poisoning Example Generation

Input: target instance t , base instance b , learning rate λ

Initialize $x: x_0 \leftarrow b$

Define: $L_p(x) = \|f(\mathbf{x}) - f(\mathbf{t})\|^2$

for $i = 1$ **to** $maxIters$ **do**

 Forward step: $\hat{x}_i = x_{i-1} - \lambda \nabla_x L_p(x_{i-1})$

 Backward step: $x_i = (\hat{x}_i + \lambda \beta b) / (1 + \beta \lambda)$

end for

Poisoning Attacks on Transfer Learning

We consider two **transfer learning** settings for experiments

- 1 The adversary attacks a pretrained InceptionV3 network under the scenario where the weights of **all layers excluding the last are frozen**.
- 2 The adversary attacks an AlexNet architecture modified for the CIFAR-10 dataset under the scenario where **all layers are trained**.

A One-shot Kill Attack

We leverage InceptionV3 as a feature extractor and **retrain its final layer** weights to classify between dogs and fish.

- In this setting, a **one-shot kill** attack is possible; by adding just one poison instance to the training set.

A One-shot Kill Attack

We leverage InceptionV3 as a feature extractor and **retrain its final layer** weights to classify between dogs and fish.

- In this setting, a **one-shot kill** attack is possible; by adding just one poison instance to the training set.

Setup

- Training set
 - 900 instances from each class
- Test set
 - 698 instances for the dog class and 401 instances for the fish class.
- Select both target and base instances from the test set and craft a poison instance
- $maxItrs = 1000$
- $\beta = 0.25 \times 2048^2 / (dim_{base})^2$ where dim_{base} is dimension of the base instance.
- Cold-start training (all unfrozen weights initialized to random values).
- **Adam optimizer** with learning rate of 0.01 to train the poisoned network for 100 epochs.

The results

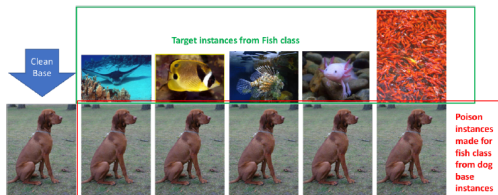
The experiment is performed 1099 times (one time for each test sample)

- Attack success rate of 100%
- Clean Accuracy 99.5% (without poisoning sample)
 - Dropping by an average of 0.2%, when the model is trained on poisoned dataset

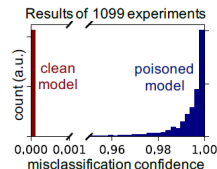
The results

The experiment is performed 1099 times (one time for each test sample)

- Attack success rate of 100%
- Clean Accuracy 99.5% (without poisoning sample)
 - Dropping by an average of 0.2%, when the model is trained on poisoned dataset



(a) Sample target and poison instances.



(b) Incorrect class's probability histogram predicted for the target image by the clean (dark red) and poisoned (dark blue) models. When trained on a poisoned dataset, the target instances not only get misclassified; they get misclassified with high confidence.

Figure 1: Transfer learning poisoning attack